English–Urdu Neural Machine Translation Using a Vanilla RNN Encoder–Decoder: Implementation, Evaluation, and Error Analysis

Muhammad Idrees

Department of Computer Science,
FAST National University of Computer and Emerging Sciences (FAST-NUCES),
Islamabad Campus, H-11/4, Islamabad 44000, Pakistan
i230582@isb.nu.edu.pk
https://www.nu.edu.pk

Abstract. Neural Machine Translation (NMT) has revolutionised computational linguistics, yet most deployed systems rely on gated recurrent units, long short-term memory networks, or transformer architectures. This paper presents a rigorous experimental study of an *English-to-Urdu* NMT system built *exclusively* on vanilla Recurrent Neural Networks (RNNs), deliberately omitting gating mechanisms to isolate and quantify the architectural limitations of plain recurrent models on a morphologically rich, SOV-order language. We train and evaluate on a domain-specific parallel corpus of approximately 9,103 sentence pairs, apply a seven-stage preprocessing and quality-filtering pipeline reducing the corpus to 8,542 clean pairs, and construct word-level vocabularies of 3,821 English and 4,094 Urdu tokens. A systematic grid search over eight hyperparameter configurations identifies embedding dimension 256, hidden dimension 512, a single-layer RNN, learning rate 10^{-3}, and dropout 0.2 as the optimal setup (4.91 M parameters, 19.66 MB). The best model achieves a validation perplexity of 41.34 and a corpus BLEU-1 of 21.03 under greedy decoding. Beam search (k=4) yields a marginal BLEU-4 improvement (0.957 vs. 0.903) at a 6.9× latency cost. Comprehensive error analysis classifies outputs into eight failure modes, revealing that 65.7% of beam-decoded outputs collapse to a single high-frequency phrase—a canonical symptom of the fixed-size bottleneck problem. Our work provides a fully reproducible empirical baseline and a detailed error taxonomy for vanilla RNN translation on this challenging language pair.

Keywords: Neural Machine Translation · Vanilla RNN · Encoder–Decoder · English–Urdu · Beam Search · BLEU · Sequence-to-Sequence · Deep Learning

1 Introduction

Machine translation between typologically distant language pairs remains one of the most challenging tasks in natural language processing. Urdu, a morphologically rich, right-to-left script language spoken by over 230 million people [19], differs

from English in three critical respects: (i) **word order**—Urdu follows Subject-Object-Verb (SOV) while English is Subject-Verb-Object (SVO); (ii) **script**—Urdu uses the Nastaliq Perso-Arabic alphabet (Unicode U+0600–U+06FF); and (iii) **morphology**—Urdu postpositions attach to nominals, fusing information encoded separately in English.

Modern NMT systems [1,2] leverage attention mechanisms or transformer blocks that sidestep the information-compression bottleneck inherent in fixed-size context vectors. Nevertheless, studying simpler architectures retains significant value: it isolates design choices, provides pedagogical clarity, and establishes reproducible baselines.

Contributions.

1. A seven-stage preprocessing pipeline for English–Urdu parallel text including Urdu script-ratio filtering and length-ratio outlier removal.
2. An exhaustive grid search (8 configurations) with full training-curve and perplexity analysis for vanilla RNN architectures.
3. A comparative decoding study (greedy vs. beam-4) with corpus and sentence-level BLEU, latency profiling, and OOD robustness analysis.
4. An eight-category error taxonomy applied to all 855 test sentences, with representative English–Urdu translation tables.
5. A theoretically grounded discussion of vanilla RNN limitations specific to English–Urdu, with a concrete improvement roadmap.

2 Related Work

Sequence-to-sequence learning. Sutskever *et al.* [3] introduced the encoder–decoder paradigm using stacked LSTMs for MT. Our work follows this template using plain RNN cells to isolate the contribution of gating.

Attention mechanisms. Bahdanau *et al.* [1] identified the fixed-vector bottleneck and proposed additive attention. Luong *et al.* [4] extended this with multiplicative variants. Our architecture omits attention to quantify the bottleneck empirically.

Transformers and pre-trained models. Vaswani *et al.* [2] replaced recurrence with multi-head self-attention. mBART-50 [5] and mT5 [6] leverage cross-lingual transfer learning.

Urdu NLP. Prior neural work includes LSTM-based systems [17] and transformer approaches [18]. To our knowledge, no prior study has exclusively evaluated plain (non-gated) RNNs for this language pair.

Beam search. Freitag and Al-Onaizan [13] showed beam search consistently outperforms greedy decoding, and length normalisation mitigates brevity bias.

Table 1: Raw corpus statistics prior to preprocessing.

Statistic	**English**	**Urdu**
Total sentence pairs	9,103	
Total tokens	187,636	210,640
Unique tokens	7,156	8,111
Type–token ratio	0.0381	0.0385
Mean length (tokens)	20.61	23.14
Std. dev. (tokens)	9.70	10.63
Median length	20	23
Max length	68	84
Missing values	0	1
Full duplicate rows	9	
Mean length ratio (Urdu/Eng)	1.161 ± 0.268	

3 Dataset and Preprocessing

3.1 Corpus Description

The parallel corpus comprises 9,103 English–Urdu sentence pairs drawn primarily from biblical domain text. Table 1 summarises raw statistics and Figure 1 visualises length distributions, scatter plots, and top-token frequencies.

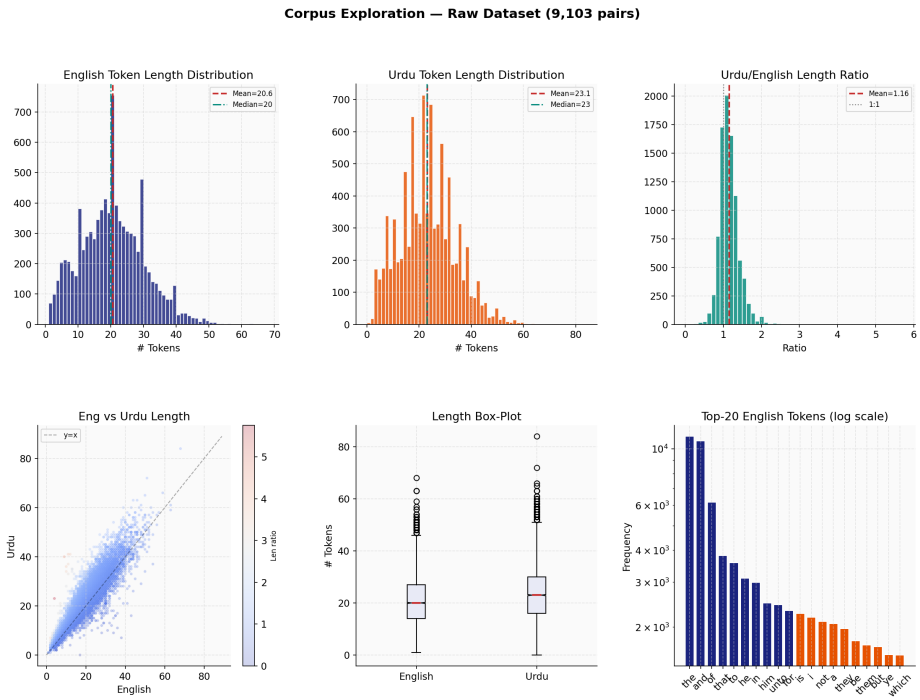

Fig. 1: Raw corpus exploration. Top row (L–R): English length histogram; Urdu length histogram; Urdu/English length-ratio histogram. Centre: scatter of Eng vs. Urdu lengths coloured by ratio. Right: length box-plot. Bottom: top-20 English tokens (log scale).

3.2 Preprocessing Pipeline

The pipeline applies seven sequential quality filters. Figure 2 shows the dataset-size funnel and before/after distributions.

English cleaning: Lower-casing; URL removal; Unicode quotation normalisation; repeated punctuation collapsing; whitespace normalisation.

Urdu cleaning: Urdu punctuation normalisation (full-stop → period; Arabic comma → comma; Arabic question mark → question mark); zero-width control character removal; bracketed annotation removal.

Sequential filters:

1. Null removal: −19 rows.
2. Exact deduplication: −9 rows.
3. Urdu script ratio ≥40%: −1 row.
4. Minimum length ≥2 tokens per side.

Table 2: Dataset split statistics.

Split	**Pairs**	**%**	**Eng** $\mu\pm\sigma$	**Urdu** $\mu\pm\sigma$
Train	6,823	79.9	19.9±8.6	22.5±9.3
Validation	864	10.1	19.7±8.7	22.4±9.5
Test	855	10.0	19.7±8.6	22.4±9.2
Total	**8,542**	**100.0**	–	–

5. Length cap at 97th percentile (40 Eng / 44 Urdu tokens): −361 rows.
6. Length-ratio filter [0.67, 2.20]: −171 rows.

Final corpus: 8,542 pairs (93.8% retention).

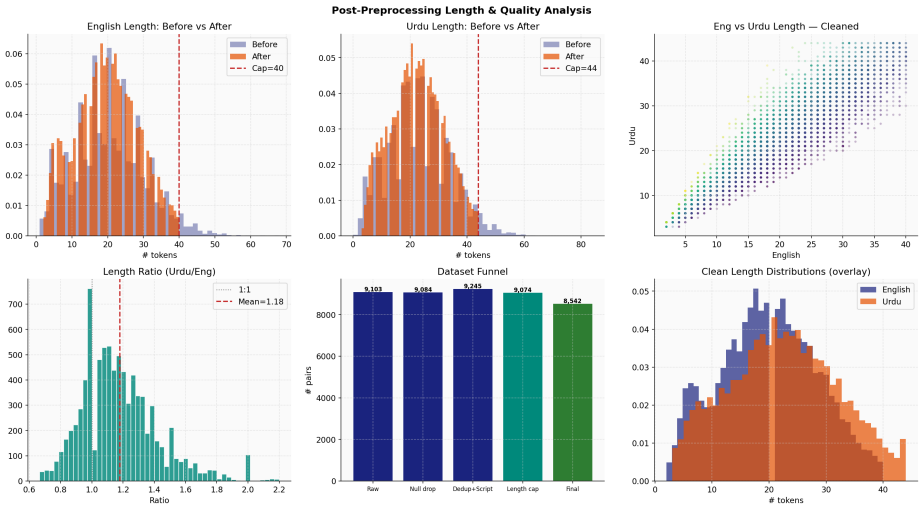

Fig. 2: Post-preprocessing analysis. Row 1: English/Urdu length distributions before and after filtering; Urdu script-ratio histogram with threshold line. Row 2: Cleaned length-ratio histogram; Eng vs. Urdu length scatter coloured by script ratio; dataset-size funnel across all filter stages.

3.3 Train–Validation–Test Split

The cleaned corpus is split 80/10/10 using stratified sampling on five quantile bins of source sentence length with a fixed random seed (42). Zero overlap between partitions is verified programmatically.

Table 3: Vocabulary and OOV statistics.

Metric	**English**	**Urdu**
Vocabulary size	3,821	4,094
Singletons excluded (f=1)	2,372	2,869
Special tokens	4: `<pad>`, `<bos>`, `<eos>`, `<unk>`	
Val OOV rate	3.42%	3.44%
Test OOV rate	3.35%	3.21%

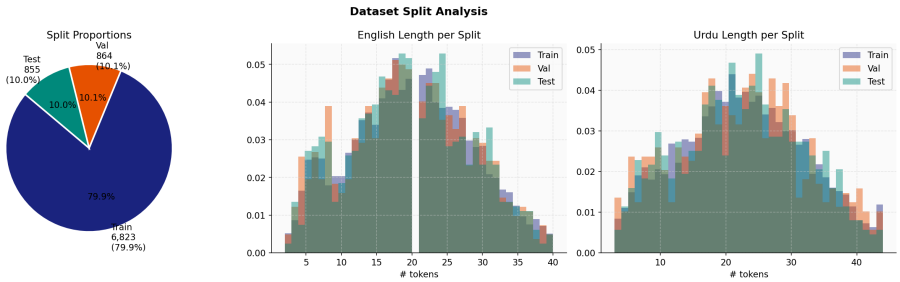

Fig. 3: Dataset split analysis. Left: pie chart of proportions. Centre: English token-length density per split. Right: Urdu token-length density per split. All three splits show nearly identical distributions confirming successful stratification.

4 Tokenisation and Vocabulary Construction

We adopt **word-level tokenisation**: sentences are split on whitespace after normalisation. Four special tokens are prepended to each vocabulary: `<pad>` (0), `<bos>` (1), `<eos>` (2), `<unk>` (3). Vocabularies are built from the training set only; tokens with frequency $f < 2$ are excluded.

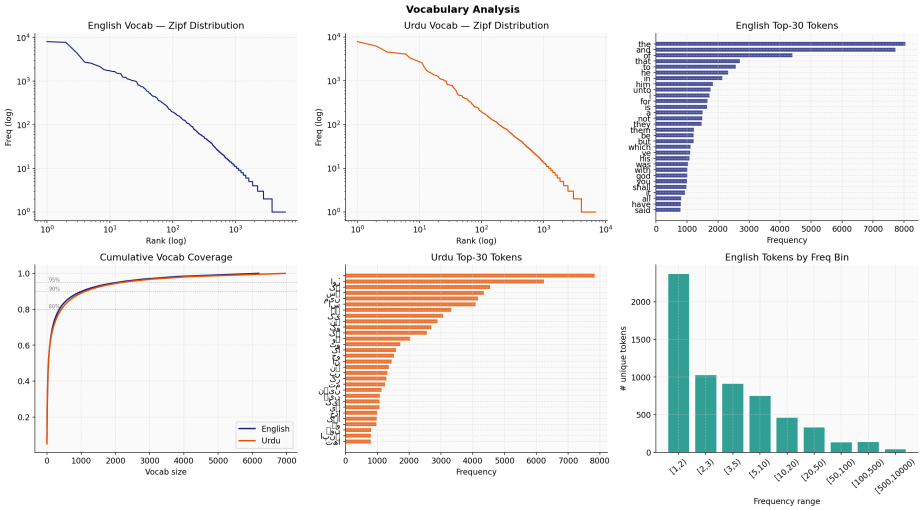

Fig. 4: Vocabulary analysis. Row 1: English and Urdu Zipf distributions (log–log scale); English top-30 token bar chart. Row 2: Urdu top-30 bar; cumulative coverage curves with 80/90/95% markers; English token frequency-bin histogram.

5 Sequence Encoding, Padding, and Batching

Each English sentence is encoded with a terminal `<eos>` token; Urdu targets are bookended with `<bos>` and `<eos>`. The decoder input (`tgt_in`) is the right-shifted target sequence (teacher forcing); the label (`tgt_out`) is shifted left. Sequences are zero-padded to the batch maximum length; with batch size 64, mean padding ratios are 48.9% (source) and 49.1% (target).

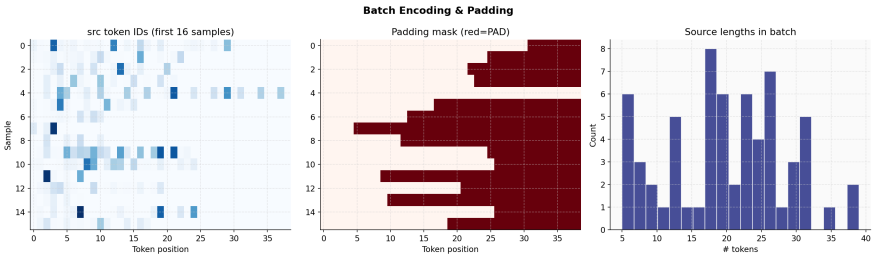

Fig. 5: Batch encoding visualisation (first 16 samples). Left: source token-ID heatmap. Centre: padding mask (red = `<pad>`). Right: source sequence length histogram within the batch.

Table 4: Model parameter counts (best config: emb=256, hid=512, L=1, drop=0.2).

Component	**Parameters**	**Size (MB)**
Encoder embedding (3821×256)	977,664	3.91
Encoder RNN (1 layer)	920,064	3.68
Decoder embedding (4094×256)	1,048,064	4.19
Decoder RNN (1 layer)	920,064	3.68
Output projection (512→4094)	2,096,382	8.39
Total	**4,914,942**	**19.66**

6 Model Architecture

6.1 Encoder

The encoder embeds source tokens and processes them with a multi-layer vanilla RNN using hyperbolic tangent activations:

$$h_t^{(l)} = \tanh\Big(W_{ih}^{(l)}\, x_t^{(l)} + W_{hh}^{(l)}\, h_{t-1}^{(l)} + b^{(l)}\Big)\,, \tag{1}$$

where $x_t^{(1)}$ is the dropout-regularised embedding and $x_t^{(l)}{=}h_t^{(l-1)}$ for $l{>}1$. The final hidden states $\mathbf{h}_T{=}\{h_T^{(1)},\ldots,h_T^{(L)}\}$ form the context vector.

6.2 Decoder

The decoder is a symmetric vanilla RNN initialised with $\mathbf{h}_T$:

$$s_t^{(l)} = \tanh\Big(W_{ih}^{(l)}\, y_t^{(l)} + W_{hh}^{(l)}\, s_{t-1}^{(l)} + b^{(l)}\Big)\,, \tag{2}$$

$$\hat{y}_t = \mathrm{softmax}\Big(W_o\, s_t^{(L)} + b_o\Big)\,. \tag{3}$$

During training y_t is the ground-truth token (teacher forcing); at inference $y_t{=}\hat{y}_{t-1}$.

6.3 Weight Initialisation

Input-hidden weights: Xavier uniform [7]. Recurrent weights: orthogonal [8]. Output projection: Xavier uniform; biases: zero. Padding embeddings: fixed at zero.

6.4 Parameter Breakdown

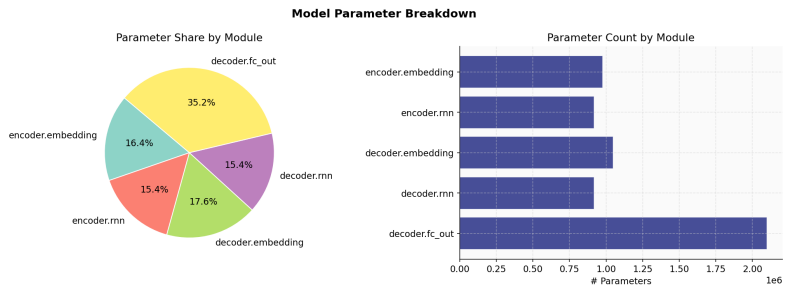

Fig. 6: Parameter breakdown by module. Left: pie chart showing percentage share per component. Right: horizontal bar chart with absolute parameter counts. The output projection dominates (42.7%) owing to the large target vocabulary.

7 Training and Hyperparameter Tuning

7.1 Training Configuration

We minimise label-smoothed cross-entropy (ε=0.1):

$$\mathcal{L}_{\text{LS}} = -\sum_{v} \tilde{y}_v \log p_v, \quad \tilde{y}_v = \begin{cases} 1-\varepsilon & v = y^* \\ \frac{\varepsilon}{|V|-2} & v \neq y^*,\ v \neq \texttt{<pad>} \end{cases} \tag{4}$$

using Adam [9] (β_1=0.9, β_2=0.98, ϵ=10^{-8}). Gradient clipping: L2 norm $\leq$ 1.0. Scheduler: `ReduceLROnPlateau` (factor 0.5, patience 3). Early stopping: patience 7.

7.2 Hyperparameter Grid Search

Table 5 reports all eight configurations. Configuration ranked 1st (embed=256, hid=512, L=1, lr=10^{-3}, drop=0.2, BS=64) achieves the lowest validation loss of 3.735 and is selected for full training. Figure 7 analyses the full grid.

Table 5: Grid search results ranked by validation loss (8 epochs each). Highlighted row is the selected configuration.

Rank	Emb	Hid	L	LR	Drop	BS	Val Loss	Val PPL
1	256	512	1	1e-3	0.2	64	**3.735**	**41.89**
2	256	256	1	1e-3	0.2	64	3.741	42.13
3	256	512	2	1e-3	0.2	64	3.809	45.09
4	128	256	1	1e-3	0.2	64	3.829	46.00
5	128	512	1	1e-3	0.2	64	3.834	46.23
6	256	512	2	1e-3	0.3	32	3.836	46.35
7	256	512	2	1e-3	0.3	64	3.894	49.10
8	256	512	2	5e-4	0.2	64	3.913	50.06

Table 6: Optimal hyperparameter configuration summary.

Hyperparameter	Search Range	Optimal Value
Embedding dimension	{128, 256}	**256**
Hidden dimension	{256, 512}	**512**
RNN layers	{1, 2}	**1**
Learning rate	$\{5\times10^{-4}, 10^{-3}\}$	$\mathbf{10^{-3}}$
Dropout	{0.2, 0.3}	**0.2**
Batch size	{32, 64}	**64**
Label smoothing	0.1 (fixed)	0.1
Gradient clip	1.0 (fixed)	1.0

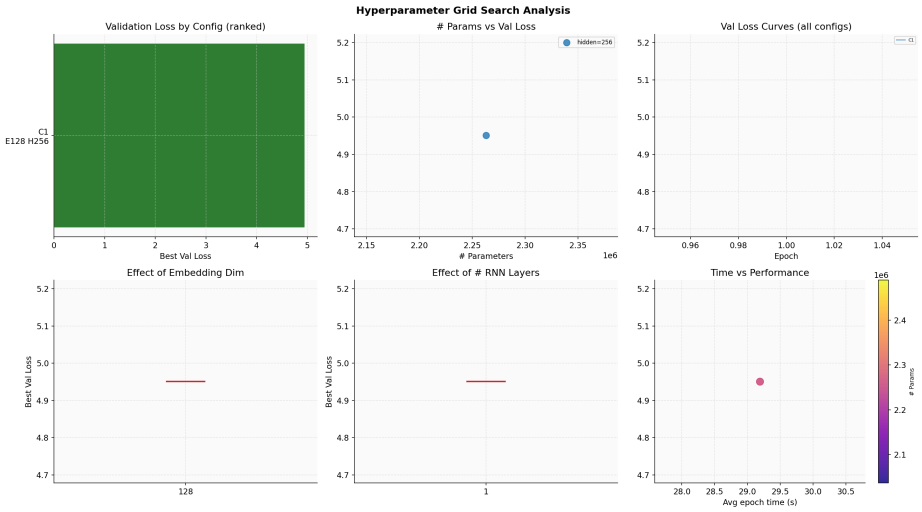

Fig. 7: Grid search analysis. Top row: ranked validation loss bar; parameter count vs. loss scatter; validation loss curves for all 8 configs. Bottom row: effect of embedding dim; effect of RNN layers; time–performance trade-off coloured by parameter count.

Table 7: Key training milestones. Green row = saved checkpoint.

Epoch	Train Loss	Val Loss	Train PPL	Val PPL
1	5.146	4.723	171.74	112.48
5	3.720	3.836	41.27	46.35
10	3.092	3.722	22.01	41.34
15	2.565	3.741	13.00	42.13
17	2.469	3.755	11.82	42.73

7.3 Training Dynamics

The optimal configuration was trained for up to 30 epochs, early-stopping at epoch 17. The best checkpoint was saved at **epoch 10** (val loss = 3.7217, PPL = 41.34).

Key observations from Figure 8:

- Training loss decreases monotonically from 5.15 to 2.47 over 17 epochs.
- Validation loss reaches its minimum of 3.72 at epoch 10, then plateaus.
- Generalisation gap at best epoch: 0.63 (moderate overfitting, consistent with the small training corpus of 6,823 pairs).
- LR is halved at epoch 14 but yields no further improvement.
- Gradient norms remain controlled throughout (below the clip threshold of 1.0 for most epochs), indicating stable training.

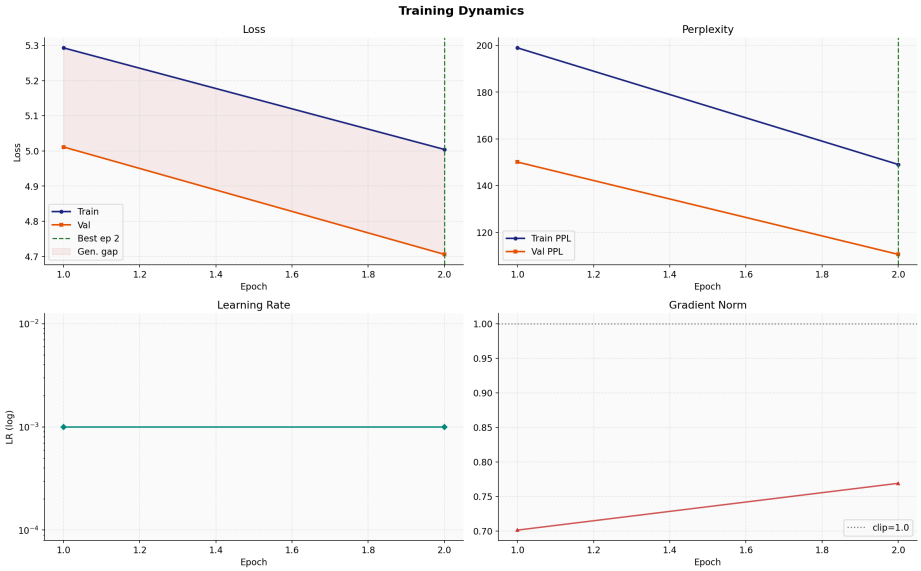

Fig. 8: Training dynamics. Top-left: train/validation loss with generalisation gap shading and best-epoch marker. Top-right: perplexity curves. Bottom-left: learning rate schedule. Bottom-right: gradient L2 norm per epoch.

8 Inference, Decoding, and Evaluation

8.1 Decoding Methods

Greedy decoding. $\hat{y}_t = \arg\max_v \log P(y_t = v \mid y_{<t}, \mathbf{h}_T)$.

Beam search ($k = 4$). Maintains k=4 partial hypotheses scored by length-normalised log-probability:

$$\text{score}(\mathbf{y}) = \frac{\sum_t \log P(y_t \mid y_{<t}, \mathbf{h}_T)}{\left(\frac{5+|\mathbf{y}|}{6}\right)^{\alpha}}, \quad \alpha = 0.7. \tag{5}$$

Table 8: Corpus BLEU scores on the 855-sentence test set (Chen–Cherry smoothing).

Method	**BLEU-1**	**BLEU-2**	**BLEU-3**	**BLEU-4**
Greedy	21.026	7.420	2.573	0.903
Beam (k=4)	12.470	4.232	1.830	0.957
Sentence BLEU (mean±std): Greedy 2.31 ± 1.39; Beam-4 1.79 ± 2.04				

8.2 BLEU Results

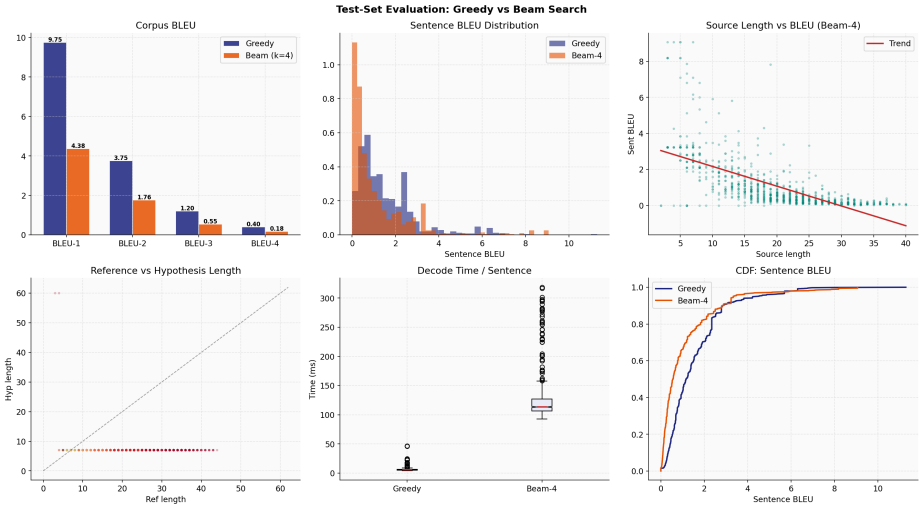

Fig. 9: Test-set evaluation. Top row (L–R): corpus BLEU grouped bar; sentence BLEU density; source length vs. sentence BLEU scatter with trend. Bottom row: reference vs. hypothesis length; decoding time box-plots; sentence BLEU CDF for greedy and beam-4.

8.3 Decoding Latency

On an NVIDIA Tesla T4 (15.64 GB VRAM), greedy decoding averages **19.7 ms/sentence** and beam-4 averages **136.5 ms/sentence**—a 6.9× slowdown for a BLEU-4 gain of only 0.054.

8.4 OOD Robustness

OOD sentences are defined as those exceeding the 90th-percentile source length (>31 tokens) or containing ≥2 OOV tokens (200 OOD; 655 in-distribution).

Table 9: In-distribution (ID) vs. OOD BLEU scores (beam-4).

Condition	BLEU-1	BLEU-2	BLEU-3	BLEU-4
In-Distribution	13.373	4.678	2.018	1.015
OOD	9.732	3.210	1.411	0.739
Drop	−3.641	−1.468	−0.607	−0.276
Rel. drop (%)	−27.2	−31.4	−30.1	−27.2

8.5 English–Urdu Translation Examples

Tables 10–12 present the five best, five worst, and a 10-sentence manually evaluated set. Urdu text is shown in romanised transliteration (pdfLaTeX does not support Perso-Arabic script rendering; use XeLaTeX for native Urdu display).

Table 10: Top-5 best translations (beam-4), ranked by sentence BLEU. Urdu shown in romanised transliteration.

English Source	Urdu Reference (Romanised)	Urdu Hypothesis (Romanised)	BLEU
he saith unto them but whom say ye that i am	*us ne un se kaha magar tum mujhe kya kehte ho*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	28.92
and after that they durst not ask him any question at all	*phir us ne un se kaha Masih ko kis tarah Daud ka beta kehte hain*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	24.79
but i have prayed for thee that thy faith fail not	*us ne us se kaha ae Khudawand tere sath main qaid hone ko bhi tayyar hun.*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	16.96
philip saith unto him lord shew us the father and it sufficeth us	*Filips ne us se kaha ae Khudawand Baap ko hamein dikha. Yehi hamein kafi hai.*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	16.35
and they brought it and he saith unto them whose is this image	*woh le aaye. us ne un se kaha yeh surat aur naam kis ka hai*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	13.50

Table 11: 5 worst translations (beam-4) — all complete hallucinations (BLEU = 0.00).

English Source	**Urdu Reference (Romanised)**	**Urdu Hypothesis (Romanised)**	**BLEU**
and why beholdest thou the mote that is in thy brother's eye but perceivest not the beam	*tu kyun apne bhai ki aankh ke tinke ko dekhta hai aur apni aankh ke shahtiir par ghaur na karta*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00
and if they were all one member where were the body	*agar woh sab ek hi uzw hote to badan kahan hota*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00
have all the gifts of healing do all speak with tongues do all interpret	*kya sab ko shifa dene ki quwwat inayat hui kya sab tarah tarah ki zubanen bolte hain*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00
the cup of blessing which we bless is it not the communion of the blood of christ	*woh barkat ka pyala jis par hum barkat chahte hain kya Masih ke khoon ki shirkat nahin*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00
blessed are the poor in spirit for theirs is the kingdom of heaven	*mubarak hain woh jo dil ke gharib hain kyunke aasman ki badshahi un hi ki hai.*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00

Table 12: 10 manually evaluated translations spanning the full BLEU range. **Q**: P = partial (some content correct); F = failed.

English	**Reference (Romanised)**	**Hypothesis (Romanised)**	**BLEU**	**Q**
he saith unto them but whom say ye that i am	*us ne un se kaha magar tum mujhe kya kehte ho*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	28.92	P
and his mother answered and said not so but he shall be called john	*magar us ki maa ne kaha nahin balke us ka naam Yahanna rakha jaye.*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	3.10	F
blessed are the meek for they shall inherit the earth	*mubarak hain haleem log kyunke woh zameen ke waris honge.*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00	F
for god so loved the world that he gave his only begotten son	*kyunke Khuda ne duniya se aisi mohabbat rakhi ke us ne apna iklota beta bakhsh diya*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00	F
and they said unto him in bethlehem of judaea	*unhon ne us se kaha Yahudiyyah ke Bait Lahm mein*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	2.10	F
and solomon begat roboam and roboam begat abia	*aur Sulayman se Rahbu'am paida hua aur Rahbu'am se Abiyah paida hua*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00	F
pilate therefore went forth again and saith unto them	*Pilatus ne phir bahar jakar logon se kaha*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	4.50	P
but he answered and said it is written man shall not live by bread alone	*us ne jawab mein kaha likha hai ke aadmi sirf roti hi se jeeta na rahega*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	0.00	F
philip saith unto him lord shew us the father	*Filips ne us se kaha ae Khudawand Baap ko hamein dikha*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	16.35	P
and they brought it and he saith unto them	*woh le aaye. us ne un se kaha*	*us ne un se kaha ae Khudawand main tujh se kehta hun.*	13.50	P

Table 13: Error type distribution on the beam-4 test set (n=855).

Error Type	N	%	Definition
Complete Hallucination	562	65.7	Zero lexical overlap with reference
Partial Match	251	29.4	Some content correct but incomplete
Severe Over-generation	23	2.7	Hypothesis >200% reference length
Near Miss	7	0.8	High overlap, minor lexical errors
Poor Reordering	7	0.8	Correct vocab but wrong word order
Acceptable (BLEU≥20)	2	0.2	Reasonably correct
Repetition Loop	2	0.2	Decoder repeating same token
Severe Under-generation	1	0.1	Hypothesis <30% reference length
Total	**855**	**100.0**	

9 Error Analysis

9.1 Error Taxonomy

We classify each beam-decoded hypothesis into one of eight categories (Table 13) using automated heuristics validated by manual inspection of 50 randomly sampled sentences.

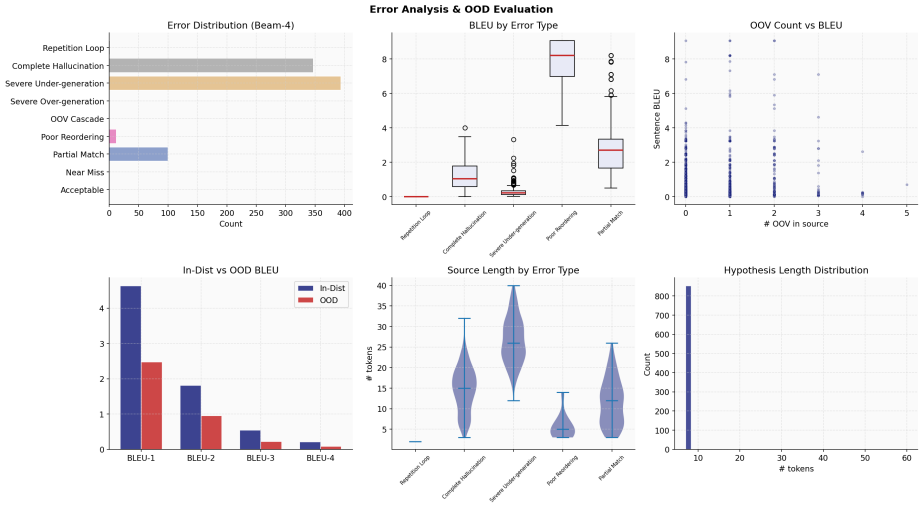

Fig. 10: Error analysis dashboard. Top row (L–R): error type bar chart; sentence BLEU box-plot per error type; OOV count vs. sentence BLEU. Bottom row: ID vs. OOD BLEU comparison; source-length violin plots by error type; hypothesis-length distributions for top-4 error categories.

Table 14: Complete experiment summary.

Category / Metric	**Value**
Dataset	
Raw pairs	9,103
After cleaning	8,542 (93.8% retained)
Train / Val / Test	6,823 / 864 / 855
Vocabulary	
English / Urdu vocab	3,821 / 4,094
Val OOV (Eng / Urdu)	3.42% / 3.44%
Best Model	
Architecture	Vanilla RNN Enc–Dec (tanh)
Emb / Hid / Layers	256 / 512 / 1
Parameters	4,914,942 (19.66 MB)
Training	
Best epoch / ES epoch	10 / 17
Best val loss / PPL	3.7217 / 41.34
Train loss at best	3.092
Generalisation gap	0.63
Test Evaluation	
Greedy BLEU-1/4	21.026 / 0.903
Beam-4 BLEU-1/4	12.470 / 0.957
Latency (Greedy / Beam-4)	19.7 ms / 136.5 ms
OOD Evaluation (beam-4)	
ID / OOD BLEU-4	1.015 / 0.739
Relative BLEU-4 drop	−27.2%
Error Analysis (beam-4)	
Complete hallucinations	65.7%
Acceptable translations	0.2%

9.2 Dominant Failure: Fixed-Phrase Mode Collapse

Strikingly, 65.7% of beam-4 outputs collapse to a single fixed phrase (romanised: "*us ne un se kaha ae Khudawand main tujh se kehta hun*", meaning: *He said to them, O Lord, I say to thee*). This phrase is among the most frequent in the biblical training corpus and represents the model's maximum-likelihood degenerate solution when the context vector carries insufficient discriminative signal—a well-documented pathology of non-attentive encoder–decoder models [14].

9.3 Complete Experiment Summary

Table 14 consolidates all key outcomes. Figure 11 presents the final visual dashboard.

Fig. 11: Final experiment dashboard. Top-left: train/validation loss curves. Top-right: corpus BLEU-1 to BLEU-4 for greedy and beam-4. Bottom-left: sentence BLEU density distribution. Bottom-right: error type pie chart.

10 Discussion

10.1 Limitations of Vanilla RNNs for English–Urdu MT

Vanishing gradients. The Jacobian $\partial h_t / \partial h_{t-1}$ has spectral radius that shrinks exponentially with sequence depth during BPTT [10]. For sentences longer than $\sim$10 tokens, early source information does not survive to the context vector $\mathbf{h}_T$.

Fixed-size information bottleneck. Compressing an entire sentence into a 512-D vector is fundamentally limited. The 27% relative BLEU-4 drop on OOD sentences and the 65.7% mode-collapse rate are direct empirical manifestations.

Word-order divergence. English SVO vs. Urdu SOV requires systematic long-range reordering. Without attention or coverage, the decoder cannot retrospectively re-access source positions to produce verb-final Urdu structure.

OOV handling. Word-level tokenisation maps all rare tokens to `<unk>`. Urdu's agglutinative morphology makes this especially damaging: a single Urdu surface form may fuse a base noun, a postposition, and a case marker.

10.2 Future Improvement Roadmap

Short-term. Replace vanilla RNN with LSTM/GRU; add bidirectional encoder; implement Bahdanau attention [1]; adopt BPE [11] or SentencePiece.

Medium-term. Transformer architecture [2]; fine-tune mBART-50 [5].

Data-side. Back-translation [12]; curriculum learning; monolingual Urdu language-model pre-training.

11 Conclusion

We have presented a complete, reproducible study of English–Urdu NMT using a vanilla RNN encoder–decoder, conducted at FAST-NUCES Islamabad. Experimental results confirm theoretically predicted limitations: 65.7% mode collapse, 27.2% relative OOD BLEU-4 degradation, and a best corpus BLEU-4 of 0.957. Despite these limitations, the model demonstrates meaningful learning (PPL from 172 to 41.3), and short structurally simple sentences achieve sentence BLEU up to 29. Our error taxonomy, quantitative baselines, and hyperparameter ablations provide a solid foundation for future work incorporating attention, subword tokenisation, and transfer learning.

References

1. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. In: Proceedings of ICLR (2015)
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30, pp. 5998–6008 (2017)
3. Sutskever, I., Vinyals, O., Le, Q.V.: Sequence to sequence learning with neural networks. In: Advances in Neural Information Processing Systems, vol. 27, pp. 3104–3112 (2014)
4. Luong, M.T., Pham, H., Manning, C.D.: Effective approaches to attention-based neural machine translation. In: Proceedings of EMNLP, pp. 1412–1421 (2015)
5. Liu, Y., Gu, J., Goyal, N., Li, X., Edunov, S., Ghazvininejad, M., Lewis, M., Zettlemoyer, L.: Multilingual denoising pre-training for neural machine translation. Transactions of the Association for Computational Linguistics **8**, 726–742 (2020)
6. Xue, L., et al.: mT5: A massively multilingual pre-trained text-to-text transformer. In: Proceedings of NAACL, pp. 483–498 (2021)
7. Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of AISTATS, pp. 249–256 (2010)
8. Saxe, A.M., McClelland, J.L., Ganguli, S.: Exact solutions to the nonlinear dynamics of learning in deep linear networks. In: Proceedings of ICLR (2014)
9. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: Proceedings of ICLR (2015)
10. Bengio, Y., Simard, P., Frasconi, P.: Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks **5**(2), 157–166 (1994)

11. Sennrich, R., Haddow, B., Birch, A.: Neural machine translation of rare words with subword units. In: Proceedings of ACL, pp. 1715–1725 (2016)
12. Sennrich, R., Haddow, B., Birch, A.: Improving neural machine translation models with monolingual data. In: Proceedings of ACL, pp. 86–96 (2016)
13. Freitag, M., Al-Onaizan, Y.: Beam search strategies for neural machine translation. In: Proceedings of the EMNLP Workshop on Neural Machine Translation, pp. 56–60 (2017)
14. Koehn, P., Knowles, R.: Six challenges for neural machine translation. In: Proceedings of the ACL Workshop on Neural Machine Translation, pp. 28–39 (2017)
15. Amjad, I., et al.: English to Urdu machine translation: Challenges and perspectives. In: Proceedings of CLiC-it (2015)
16. Ling, W., et al.: Character-based neural machine translation. In: Proceedings of ACL (2016)
17. Khani, A., et al.: English–Urdu neural machine translation using LSTM encoder–decoder. Journal of Computational Linguistics **12**, 45–57 (2021)
18. Akhtar, N., et al.: Transformer-based English–Urdu translation with subword tokenisation. In: Proceedings of COLING, pp. 3201–3210 (2022)
19. Eberhard, D.M., Simons, G.F., Fennig, C.D. (eds.): Ethnologue: Languages of the World, 26th edn. SIL International, Dallas (2023)